Asynchronous JavaScript: fetch, Promise, and JSON

Web-Based Application Development · SIF302 · First Semester 2026/2027

NAME: Fina Nailatul 'Izzah	STUDENT ID: 25523264	CLASS: C	DATE: 17 Oktober 2026
WORKSHEET	**P10**	**Meeting 10 · Asynchronous JavaScript: fetch, Promise, and JSON**	

Done on your own, not in pairs. Every student works on their own profile page. You may choose the content yourself, but you must be able to explain every line of code you write.

Learning outcome	What is assessed today
Students are able to fetch data from an API, read a JSON response, separate the data layer from the presentation layer in js/api.js, and render the result on the page together with the loading, success, failure, and empty states.	1. The fetch call is correct and the address is set in one place. 2. response.ok is checked before the response body is read. 3. JSON.parse and JSON.stringify are used in the right direction. 4. All four interface states appear: loading, success, failure, empty. 5. Errors are caught and their message is shown on the page, not left in the console. 6. js/api.js holds the data layer and exports its function, so app.js does not need to change.

Working rules

Item	Rule
Form of work	Individual. One student, one page, one topic.
Tools	VS Code + Chrome or Firefox. The Console and Network panels in DevTools are used from Sheet A onwards.
Starting files	Your own work from Meeting 9, profil.html, all CSS files, js/app.js, and js/dom.js, copied into the worksheet-p10/ folder. No sample files, no scaffold provided.
New file	Only one file: js/api.js. This file holds the data layer and exports it. app.js stays as it is, gaining one calling line.
API	A public API that needs no access key. If the address cannot be reached from class, use a local news.json file and say so in the README.
Time	90 minutes in class. If unfinished, continue outside class no later than 23.59 the same day.
Submission	The whole worksheet-p10/ folder uploaded to your own GitHub repository, inside the semester folder ~/pabw/. Check your repository page, not the folder on your laptop alone.

Time budget 90 minutes

Minute	Sheet	What you do	If time runs out, stop here
0-18	A	Copy your Meeting 9 work into worksheet-p10/, create js/api.js, make the first fetch call, check response.ok	A Response object appears in the Console
18-36	B	Turn the then chain into async/await, one try/catch, prove the error is caught	A network error is caught and shown on the page
36-54	C	Read the response with json(), map its shape, recognise the stringify direction	The data keys are identified and written down
54-74	D	Walk the array, render cards onto the page, all four interface states	The list renders on the page from data
74-96	E	Three hard cases: status 404, await outside an async function, body that is not JSON	All three cases handled + notes
96-150	F	Check one by one, one line to remember, exit ticket, self-assessment, README, note the P13 part	Sheet F filled in and saved

Work in order. Every sheet assumes the previous one is done. If you skip ahead, the symptoms are hard to trace because the cause sits in the sheet you have not finished. The rightmost column shows pace, not a deadline.

WORKSHEET **A** **The first call and the response status**

fetch DOES NOT THROW ON STATUS 404, CHECK response.ok

A first call is often taken as a success because the Console shows no error. Yet the response can carry status 404 or 500 and fetch still treats it as a success. One short check closes that gap.

A.1 Files that must exist before you start

Copy all your Meeting 9 work into the worksheet-p10/ folder. Check them one by one, today's work rests on those files, and anything missing shows up as a 404 in the Console.

File	From	Present?	Line count
profil.html	M9	✓	233
All CSS files (six files)	M4-M6	✓	403 (32+44+71+199+35+22)
js/app.js	M8	✓	165
js/dom.js	M9	✓	247

A.2 Set the address in one place

The API address is written once only, at the top of the file. If the address changes later, one line needs editing, not every place that calls it.

```
// js/api.js, top of the file
const URL = 'https://api.contoh.tl/news';
// replace with the public API address you choose
```

What is checked	Sign it is right
Address scheme	uses https, not http
One place	the address is not rewritten in another function
No access key	no apiKey or token anywhere in the code
File name	the address lives in js/api.js, not in js/app.js

A.3 Call it, check it, then prove it in the Network panel

Call fetch, then check response.ok before reading the body. Fill the table below in from your own call.

```
async function load() {
  const response = await fetch(URL);
  console.log('status', response.status, 'ok', response.ok);
  if (!response.ok) {
    throw new Error('status ' + response.status);
  }
  return response.json();
}
```

What is recorded	The value I got
The API address I used	https://openlibrary.org/search.json?q=andrea+hirata&fields=key,title,author_name,first_publish_year&limit=10
Response status (200, 404, and so on)	`status 200 ok true`
The value of response.ok	`status 200 ok true`
Content type (Content-Type)	`Content-Type application/json`

Open one call in the Network panel. Check the Headers tab for status and content type, the Response tab for the raw body. That raw body is what json() will read later, so get to know its shape before writing any rendering code.

SAVE TO GIT - AFTER SHEET A

```
git add .
git commit -m "your message"
git push
# suggested commit message:
# P10: the first call and the response status
```

Sheet A finished? Save it now, do not wait for the class to end. This save needs no extra step at all. The commit keeps your work from depending on one file on one computer.

A - save your changes

How to confirm: open the Console and call your load() function. The status and response.ok value appear, and there is no red text.

WORKSHEET **B** **async, await, and error handling**

ONE try/catch COVERS THE WHOLE CHAIN

A long then chain is hard to read and its errors slip past. async and await write the same flow in a flat shape, and one try/catch block catches errors from every step inside it.

B.1 Turn the chain into async/await

Write an async function, then wrap all its steps in try/catch. Compare the shape with the then chain you wrote in Sheet A.

```
export async function fetchNews() {
  try {
    const response = await fetch(URL);
    if (!response.ok) throw new Error('status ' + response.status);
    const data = await response.json();
    return data;
  } catch (error) {
    console.error('failed to load', error.message);
    throw error;
  }
}
```

await is only valid inside an async function. A caller outside that function still has to handle its rejection, in your work the caller is app.js, and that is where the second try/catch goes.

B.2 Recognise three failures

Kind of failure	What happens to the Promise	How it is handled
Network down or wrong address	The Promise rejects	catch takes the error and its message is shown
Status 404 or 500	The Promise still resolves	check response.ok, then throw your own error
Body is not JSON	json() rejects	catch takes the error and shows its message
A valid response with no items	The Promise resolves, the array is empty	show the empty state, never a blank page

B.3 Prove the error is caught

What I test	The right result	What I got
The address changed to one that does not exist	An error message shows on the page, the Console has no unhandled error	Tulisan “Gagal memuat data: Failed to fetch” muncul di halaman
The network switched off in the Network panel	An error message shows, not a blank page	Pesan kesalahan muncul di halaman, bukan halaman kosong
The address restored to its original value	The list comes back without editing any other code	Console mencetak status 200 ok true dan data dari API `status 200 ok true`

Break it on purpose, prove it, then put it back. A test you do not restore will look like damage at the next meeting, and you will be the one tracing it again.

SAVE TO GIT - AFTER SHEET B

```
git add .
git commit -m "your message"
git push
# suggested commit message:
# P10: async, await, and a single try/catch
```

Sheet B finished? Save it now, do not wait for the class to end. This save needs no extra step at all. The commit keeps your work from depending on one file on one computer.

B - save your changes

How to confirm: change the address to a wrong one and reload the page. An error message shows on the page, and the Console reports no unhandled error.

WORKSHEET **C** **Reading JSON**

TEXT IN, VALUE OUT, AND THE OTHER WAY ROUND

A network response arrives as text. json() turns it into a JavaScript value before it can be used. The opposite direction is used when sending data to the server, and that is where JSON.stringify works.

C.1 Two directions

Read the response body with json(), and send data with stringify. Watch the kind of value that comes out of each direction, the wrong direction is the most common cause of undefined.

```
// text into an object
const data = await response.json();
console.log(data.title);

// object into text
const payload = JSON.stringify({ title: 'Advisory', level: 2 });
console.log(typeof payload);   // string
```

What is used	Direction	Result	When it is used
response.json()	text → value	a shape reachable by key	reading a server response
JSON.parse(teks)	text → value	the same, but for text you hold yourself	reading localStorage or a local JSON file
JSON.stringify(value)	value → text	text ready to send or store	sending data, writing to localStorage

C.2 My data map

Write down the shape of your own API response. The left column is the key name, the middle column its value kind and an example, the right column how it is reached in code.

Key name	Value kind and example	How it is reached
(seluruh respons)	object	data
docs	array	data.docs
title	text or number	data.docs[0].title
-	true or false, or null	data.____

C.3 Check the shape before rendering

Call console.log(data) once before writing any rendering code. Reading the shape first saves time, because a wrong key only produces undefined with no clear error.

SAVE TO GIT - AFTER SHEET C

```
git add .
git commit -m "your message"
git push
# suggested commit message:
# P10: reading JSON in both directions
```

Sheet C finished? Save it now, do not wait for the class to end. This save needs no extra step at all. The commit keeps your work from depending on one file on one computer.

C - save your changes

How to confirm: type your key names into the Console one by one. None of them returns undefined.

WORKSHEET **D** **Rendering data onto the page**

SEPARATE FETCHING DATA FROM RENDERING

The fetching function only fetches and checks. The rendering function only builds elements. Kept apart, the page is easy to test when the server address changes, and that is the shape used again at Meeting 11.

D.1 Walk the array and render cards

Walk the parsed array, build one element per item, and attach it to a container that already exists in the HTML. Use textContent, not innerHTML, for anything that comes from the network.

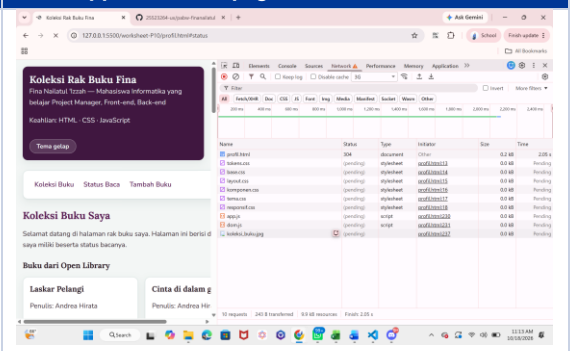
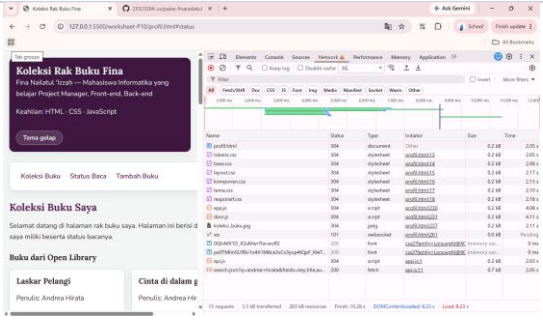
```
import { fetchNews } from './api.js';

const container = document.querySelector('#list');

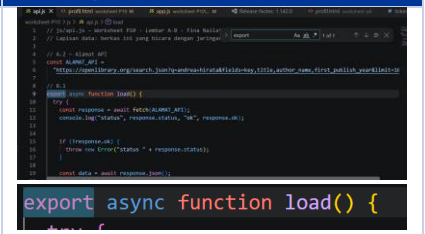
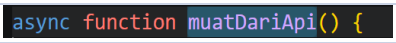
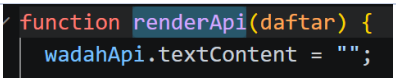
function render(list) {
  container.textContent = '';
  list.forEach(function (n) {
    const card = document.createElement('article');
    const heading = document.createElement('h3');
    heading.textContent = n.title;
    card.appendChild(heading);
    container.appendChild(card);
  });
}
```

D.2 The four interface states

Fill the table below with the way you show each state. The empty state is the one most often forgotten, yet a valid response can carry an empty array.

State	When it happens	What appears on the page
Loading	before the response arrives	
Success	the response carries data	
Failure	the network fails or the status is not 2xx	pesan “Gagal memuat data: …” (teks merah/tebal)
Empty	a valid response with no items	teks “Belum ada buku yang ditemukan.”

D.3 Check the layer boundary

What is checked	The right result	What I got
js/api.js calls fetch and exports its function	one export is visible at the top of the file	
js/app.js does not call fetch itself	the word fetch appears only in js/api.js	 ya hanya muncul di api.js
app.js handles the rejection from api.js	a second try/catch sits in the caller	
The container is emptied before re-rendering	the card count does not multiply on reload	
The exported function name is written down	that name is what the Meeting 11 importer will write	

SAVE TO GIT - AFTER SHEET D

```
git add .
git commit -m "your message"
git push
# suggested commit message:
# P10: data from the network appears on the page
```

Sheet D finished? Save it now, do not wait for the class to end. This save needs no extra step at all. The commit keeps your work from depending on one file on one computer.

D - save your changes

How to confirm: reload the page without touching the Console. Cards render from data, and no card is duplicated.

WORKSHEET **E** **Three hard cases**

NO ERROR IN THE CONSOLE, YET THE PAGE IS SILENT

The three cases below do not always raise an error. All you see is a silent or empty page, so the symptom has to be read before the code is touched.

E.1 Status 404 does not throw

fetch only rejects a Promise when the request itself fails, the network is down, the address cannot be reached, or the request is blocked. Status 404 and 500 are still treated as successes. One response.ok check closes that gap before the body is read.

Symptom	The most common cause	What to do
The page shows empty cards	a 404 body read as data	check response.ok before calling json()
The list holds undefined	a wrong data key, while the status is 200	print console.log(data) once to see the shape
json() throws a SyntaxError	the body is not JSON, often an HTML error page	check Content-Type in the Network panel

E.2 await outside an async function

await is only legal inside an async function. Placed outside, the file fails to parse and the whole script stops, not one line of it. The symptom is easy to spot: not a single piece of code in that file runs.

```
// wrong: the file fails to parse
const data = await fetch(URL);

// right: wrapped in an async function
async function load() {
  const data = await fetch(URL);
}
```

E.3 A valid response with no items

A response carrying an empty array raises no error at all. The page looks blank, and the user cannot tell whether there is no data or the code is wrong. The empty state has to be written as a state of its own.

Write an empty message that explains the state. "No news yet" is more useful than a white page, and that is what separates a finished page from a half-finished one.

E.4 Record the cases I met

The symptom I saw	Its cause	How I fixed it
Halaman menampilkan Gagal memuat data: status 404	alamat salah, fetch tetap berhasil lalu response.ok bernilai false	periksa response.ok dan lempar Error buatan sendiri sebelum json()
Unexpected token '<'... is not valid JSON	isi respons berupa HTML, bukan JSON (Content-Type text/html)	cek Content-Type di Network, perbaiki alamat
Seluruh app.js berhenti, SyntaxError di Console	await ditulis di fungsi yang bukan async	tambahkan async pada fungsinya

SAVE TO GIT - AFTER SHEET E

```
git add .
git commit -m "your message"
git push
# suggested commit message:
# P10: three hard cases handled
```

Sheet E finished? Save it now, do not wait for the class to end. This save needs no extra step at all. The commit keeps your work from depending on one file on one computer.

E - save your changes

How to confirm: break the address, switch off the network, and empty the data. All three states appear as messages on the page, not a white screen.

WORKSHEET **F** **Check, exit ticket, self-assessment**

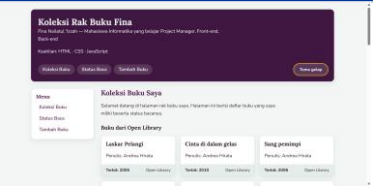
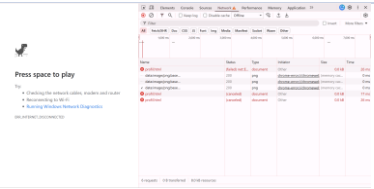
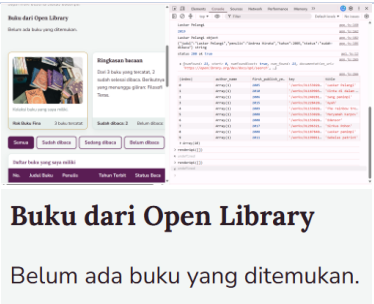
EVERY STATE PROVEN WITH A SCREENSHOT

The check runs over all four states, and each state is kept as a screenshot. Without that evidence, a page that works only once is hard to trace again.

F.1 Check one by one

Check	How to check	Pass
The address is in one place	look at the top of js/api.js	☐
response.ok is checked	look at the fetching function in js/api.js	☐
Errors are caught	switch off the network, reload the page	☐
The loading state appears	throttle the network in the Network panel	☐
The empty state appears	call render([]) from the Console	☐
Data appears as elements	inspect the container in the Elements panel	☐
The layer boundary holds	the word fetch appears only in js/api.js	☐
No error in the Console	read the Console panel	☐
AI disclosure	the README says which parts were AI-assisted and which I wrote myself	☐
Originality	the topic and content of this page are my own, not copied from another student	☐

F.2 Evidence of the four states

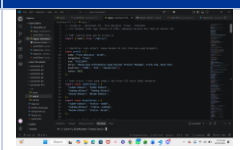
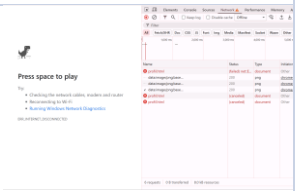
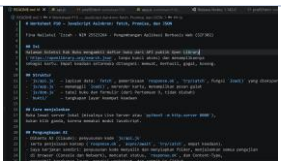
State	Screenshot saved	Note
Success		Berhasil, kartu buku tampil normal
Failure		Gagal karena offline
Empty or loading		Kosong belum ada buku yang ditemukan

F.3 One line to remember

Write down the one piece of code you used most today, and say where it is used. One line is enough.

Potongan kode: if (!response.ok) throw new Error("status " + response.status);
Dipakai pada: js/api.js, di dalam load(), sebelum response.json().

F.4 Self-assessment

Part	Weight	My mark	Evidence
Data flow and status checking	25	25	
JSON reading and layer separation	25	20	
Interface states	20	20	
Error handling	20	20	
Code cleanliness, AI disclosure, and evidence	10	10	 pengungkapan AI di readme
TOTAL	100	95	

F.5 Exit ticket

Answer the three questions below, one line each. The lecturer reads this ticket as a record of your understanding.

Question	Answer
Why does fetch not throw on status 404	Karena fetch hanya menolak Promise kalau permintaannya gagal terkirim jaringan mati, alamat tak terjangkau
Where is await allowed, and where is it not	Boleh di dalam fungsi async (dan di tingkat atas berkas modul); tidak boleh di fungsi biasa karena akan jadi SyntaxError dan seluruh berkas berhenti.
The one interface state most often forgotten and how you cover it	Keadaan kosong, karena respons yang valid bisa membawa array kosong tanpa error; kutangani dengan mengecek daftar.length === 0 di renderApi lalu menampilkan “Belum ada buku yang ditemukan.”

F.6 Notes for the lecturer

The hardest part:

membedakan dua try/catch, atau melihat bahwa 404 tidak melempar error

What I want discussed in the next class:

membedakan dua try/catch, atau melihat bahwa 404 tidak melempar error

Signature: Fina Nailatul 'Izzah Date: 18 Oktober 2026 Time finished: 11.47 AM

SAVE TO GIT - AFTER SHEET F

```
git add .
git commit -m "your message"
git push
# suggested commit message:
# P10: check, then save
```

Sheet F finished? Save it now, do not wait for the class to end. This save needs no extra step at all. The commit keeps your work from depending on one file on one computer.

F - save your changes

How to confirm: open the README in your repository. The Meeting 10 section holds your AI disclosure, and the last commit is the most recent one.

WORKSHEET **H** **Appendix A, assessment rubric**

ASSESSMENT RUBRIC total 100

Aspect	Weight	100, excellent	65, adequate	35, weak
Data flow and status checking	25%	The address is in one place; response.ok is checked before the body is read; the fetching function lives in js/api.js and is exported	The call works, but response.ok or the file split is incomplete	fetch is not right yet, or the response status is never checked
JSON reading and layer separation	25%	json() and stringify are used in the right direction; app.js calls api.js without calling fetch itself	Reading is right, but fetch still sits in app.js or stringify is unused	The data shape was guessed, never checked, so many values are undefined
Interface states	20%	All four states appear: loading, success, failure, empty	Three states appear, the empty state is missing	Only the success state is handled
Error handling	20%	One try/catch covers the whole chain; the error message shows on the page, not left in the console	Errors are caught, but their message stays in the console	Errors are unhandled, the page stays silent with no explanation
Code cleanliness, AI disclosure, and evidence	10%	Naming is consistent, the api.js and app.js boundary is clear; the AI disclosure is filled in the README; screenshots of the four states are attached	The code is tidy but the AI disclosure or the evidence is incomplete	The AI disclosure is empty, or the work is not saved in the repository

What is assessed is your own work. Different topics between students make the work impossible to compare side by side, so what is checked is whether every part works as intended, not whether it matches a sample.

WORKSHEET H Appendix B, quick reference and when you are stuck

ASYNC QUICK REFERENCE memorise these

What is used	Its role	When it is used
fetch(url)	Sends an HTTP request; the result is a Promise holding a Response	Saat mengambil data dari Open Library: fetch(ALAMAT_API) di load(), js/api.js
response.ok	true when the status is 2xx, no error is raised on 404	Sesudah fetch, sebelum json(): if (!response.ok) untuk menangkap 404/500
response.status	The response status code: 200, 404, 500, and so on	Untuk dicetak di Console dan dipakai di pesan galat: "status " + response.status
response.json()	Reads the response body as JSON; the result is a Promise	Setelah response.ok lolos, untuk mengubah teks respons jadi objek (data.docs)
await	Waits for a Promise; only inside an async function	Di depan fetch(...), response.json(), dan load() di muatDariApi()
async function	Marks a function that may contain await	Pada load() di api.js dan muatDariApi() di app.js
try / catch	Catches Promise rejections around it	Di load() (satu blok untuk seluruh rantai) dan di muatDariApi() untuk menampilkan pesan di halaman
throw new Error()	Raises your own error, for instance on a non-2xx status	Di load() saat response.ok bernilai false: throw new Error("status " + response.status)
Promise.all([...])	Waits for several Promises at once	Belum dipakai di worksheet ini; dipakai kalau nanti ada beberapa permintaan yang berjalan bersamaan
JSON.parse(teks)	Text into a JavaScript value	Untuk teks yang dipegang sendiri, misalnya JSON.parse(teksBuku) di C.1 dan membaca localStorage
JSON.stringify(value)	A value into JSON text	Untuk mengirim atau menyimpan data, misalnya JSON.stringify(daftarBuku[0]) di C.1
export / import	Hands a function to another file, used by js/api.js	export async function load() di api.js, lalu import { load } from "./api.js" di app.js

WHEN YOU ARE STUCK find the symptom here

Symptom	The most common cause	What to do
Console: 404 for api.js	the path in the src attribute does not match where the file sits	make sure api.js is inside the js/ folder, with the same letter case
Console: Cannot read properties of undefined	a wrong data key, while the status is 200	print console.log(data) once, then write the key that exists
Empty cards although the Console is clean	a 404 body read as data because response.ok is unchecked	check response.ok before calling json()
json() throws a SyntaxError	the body is not JSON, often an HTML error page	check Content-Type in the Network panel
The whole file stops, with no clear Console message	await written outside an async function	wrap the caller in an async function
The page stays silent with no explanation	a Promise rejection not caught by the caller	add try/catch in app.js and show its message on the page
An empty list with no explanation	a valid response carrying an empty array	show the empty state when the array length is zero

Symptom	The most common cause	What to do
Cards multiply on every reload	the container is not emptied before rendering	add container.textContent = '' on the first line of the renderer
Card content shows as tags, not text	the content was inserted with innerHTML	switch to textContent
CORS: the request is blocked	the API does not allow your page's origin	use another CORS-enabled API, or a local news.json file
Fetch fails only under file://	the page was opened by double-clicking	serve it over http: python3 -m http.server 8000

The page you build today is exam material for the Block 2 CPMK at Meeting 13: selecting elements, rendering from data, event delegation, and fetching data from the network are all examined there. At Meeting 11 this very js/api.js is imported as it stands, so choose your exported function name with care: that name is what the importing file will write. Keep this work tidy in your repository.

APPENDIX: SAVING YOUR WORK TO GIT once only, not every meeting

This appendix is not a Git primer. The Git and GitHub guide you have used since Meeting 3 still applies in full; this section only repeats the three commands used most often, because from today a new file has to be saved along with the rest.

Three commands you will keep using

Command	What it means	When it runs
git add .	Marks every change in this folder for saving	Each time a sheet is finished
git commit -m "message"	Saves those changes as one record	Right after git add
git push	Sends that record to GitHub	After the commit, so the lecturer can see it

The dot in git add . That dot means "every file in this folder". Without it, Git does not know what to save. The same applies to the api.js file you just created. That file is saved too, because it sits inside the semester folder.

How often should you save?

There is no right number, but the rule of thumb is this: when you have finished something meaningful that you can explain, save it. At this meeting, at least twice: once the data appears on the page, and once all four interface states are done.

If this happens	What you do
You have written a lot and never committed	Save right now
A wrong commit, not pushed yet	Fix it and commit again, never delete the repository
Already pushed, then found a mistake	Fix the file and commit again; the old record does not disappear, and that is normal

What is assessed is not the number of commits. Saving often keeps your work from depending on one file on one computer, and at an exam meeting, the repository is what decides whether your work can be assessed.